Lab 4 — Introduction to Python

Review Sheet

CENG 111 | Fall 2026

1. How Python Runs

REPL (Read–Eval–Print Loop) is the interactive Python shell, launched by running `python` (or `python3`) with no arguments in the terminal. You type one expression, and Python evaluates it and immediately prints the result.

```
>>> 3 + 4
7
>>> "hello"
'hello'
```

Script mode runs a saved `.py` file from top to bottom when passed to the interpreter in the terminal.

```
# hello.py
print("Hello, CENG 111!")
```

```
$ python hello.py
Hello, CENG 111!
```

2. Data Types

Every value in Python has a type. Types matter because they determine what operations are valid and how the values are stored in memory.

Use `type()` to inspect any value:

```
>>> type(42)
<class 'int'>
>>> type(42.0)
<class 'float'>
>>> type("42")
<class 'str'>
>>> type(True)
<class 'bool'>
```

3. Arithmetic Operators

Operator	Meaning	Example
+	Addition	3 + 4
-	Subtraction	10 - 3
*	Multiplication	3 * 4
/	True division	7 / 2
//	Floor division	7 // 2
%	Modulo (remainder)	7 % 2 → 1
**	Exponentiation	2 ** 8 → 256

When an operation mixes `int` and `float`, Python promotes the result to `float`:

```
>>> type(4 * 3.75 - 1)
<class 'float'>
```

4. Built-in Functions

4.1. Numeric functions

```
>>> abs(-7)
7
>>> abs(-3.5)
3.5
```

4.2. Base conversion

```
>>> hex(255)
'0xff'
>>> bin(9)
'0b1001'
>>> oct(8)
'0o10'
>>> int('FF', 16)     # parse hex string -> int
255
>>> int('1001', 2)    # parse binary string -> int
9
>>> int('777', 8)     # parse octal string -> int
511
```

`int(s, base)`: Converts a string `s` written in the given `base` into a standard base-10 integer. For example, `int('777', 8)` parses `'777'` as base-8 ($7\times64 + 7\times8 + 7$), returning `511`.

4.3. Characters and ASCII / Unicode

Every character has a numeric code defined by the Unicode standard (whose first 128 codes match the classic ASCII table). `ord()` converts a character to its code, and `chr()` does the reverse.

```
>>> ord('A')
65
>>> ord('a')
97
>>> chr(65)
```

```
'A'
>>> chr(ord('a') + 1)    # next character after 'a'
'b'
>>> 'a' < 'b'            # characters are compared by numeric code
True
```

5. Floating-Point Precision

Floats are stored in binary (base 2). Just as $\frac{1}{3}$ cannot be written exactly in decimal, many simple decimals—including `0.1`—cannot be represented exactly in binary.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Instead of comparing floats with `==` check whether the difference is smaller than a small tolerance (epsilon):

```
abs(a - b) < 0.00001
```

6. Strings

6.1. Escape characters

Inside a string, the backslash `\` introduces a special character:

Sequence	Meaning
`\n`	Newline (move to the next line)
`\t`	Tab (horizontal space)
`\'`	Literal single quote
`\"`	Literal double quote
`\\`	Literal backslash

```
>>> print("line1\nline2")
line1
line2
>>> print("Name:\tAlice")
Name:   Alice
```

6.2. String operations

```
>>> "ceng" * 3
'cengcengceng'
>>> "ceng" + "111"
'ceng111'
>>> len("ceng111")
7
>>> "12345".isdigit()
True
>>> "ceng".upper()
```

```
'CENG'
>>> "ceng".isalpha()
True
```

6.3. Multiline strings

Triple quotes let a string span multiple lines:

```
message = """Dear student,
Welcome to CENG 111.
Good luck!"""
print(message)
```

7. Type Casting

You can convert between types with the built-in constructors `int()`, `float()`, `str()`, and `bool()`.

```
>>> int(12.9)        # truncates towards zero (does NOT round)
12
>>> int(-12.9)
-12
>>> float(7)
7.0
>>> str(42)
'42'
>>> int("42")
42
>>> bool(0)
False
>>> bool(1)
True
>>> bool("")         # empty string is false
False
```

8. Variables

A variable is a name that refers to a value. Python is *dynamically typed*: variables do not have fixed types and can be reassigned to values of any type.

```
>>> x = 10
>>> type(x)
<class 'int'>
>>> x = 3.14         # same variable, new value -- type changes
>>> type(x)
<class 'float'>
```

Referencing a name before assigning it raises a **NameError**:

```
>>> print(y)
NameError: name 'y' is not defined
```

Naming rules & conventions: Names may contain letters, digits, and underscores (_), cannot start with a digit, and are case-sensitive. You can use `lower_case_with_underscores`.

```
first_number  = 10
second_number = 20
result = str(first_number) + str(second_number)    # "1020"
```

9. Input and Output

9.1. `print()`

`print()` accepts any number of arguments separated by commas; it inserts a space between them and a newline at the end.

```
>>> print(5, '+', 4, 'is', 9)
5 + 4 is 9
>>> print("x =", 42, "\t(the answer)")
x = 42  (the answer)
```

9.2. `input()`

`input()` always returns a **string**, regardless of what the user typed. Cast it to get a number:

```
n = int(input("Enter a number: "))
x = float(input("Enter a decimal: "))
```

9.3. `eval()`

`eval()` evaluates a string as a Python expression and returns the result:

```
>>> eval("5 + 7 * 2")
19
>>> eval("3.14 * 2")
6.28
```

Combining `eval()` with `input()` lets the user type a mathematical expression directly:

```
result = eval(input("Expression: "))
```

10. Practice Exercises

Write each script in a `.py` file and run it from the terminal (no solutions provided).

1. `compare.py` — Read two math expressions from the user (hint: `eval(input(...))`) and print `True` if their results are equal within an epsilon of `0.000001`, or `False` otherwise.

```
$ python compare.py
expression 1: 0.1 + 0.2
expression 2: 0.6 / 2
True
```

2. `max.py` (Bonus) — Read two integers from the user and print the larger one *without* using `if` statements or `max()` (use only the operators and functions from this lab).